E-Grocery Project Defense Reviewer

1. Project Overview

E-Grocery ni Aling Bebang — an ASP.NET Core MVC web app where users browse grocery products, add to cart, place orders, and admins manage products.

Tech Stack:

- ASP.NET Core MVC
- Entity Framework Core (EF Core)
- SQL Server LocalDB
- Razor Views with Tailwind CSS

---

2. Architecture (MVC Pattern)

Layer	What It Does	Our Files
Model	Data classes + validation rules	`User.cs`, `Product.cs`, `CartItem.cs`, `Order.cs`, `RegisterViewModel.cs`, `LoginViewModel.cs`, `ProductCreateViewModel.cs`
View	Displays UI (HTML)	`.cshtml` files in `Views/`
Controller	Handles requests, talks to DB	`AccountController.cs`, `StoreController.cs`, `AdminController.cs`
DbContext	Bridge between code and database	`AppDbContext.cs`

---

3. Database & Session

Connection String

```
"Server=(localdb)\\mssqllocaldb;Database=EGroceryDB;TrustServerCertificate=True;"
```

Tables in `AppDbContext`

- `Users` — registered accounts + profile pictures
- `Products` — grocery items
- `CartItems` — shopping cart entries
- `Orders` — completed transactions

Session (Login Tracking)

ASP.NET Session remembers the logged-in user across page visits:

```
// On login success:
HttpContext.Session.SetString("UserEmail", user.Email);

// To check if logged in:
HttpContext.Session.GetString("UserEmail");

// On logout:
HttpContext.Session.Remove("UserEmail");
```

- If session has an email → show "My Account" and "Logout"
- If session is empty → show "Login" and "Register"
- Session expires after 20 minutes of inactivity

Migrations (How the Database Was Created)

Migrations are C# code files that tell EF Core how to build the database schema:

```
Add-Migration InitialCreate       → creates a migration file
Update-Database                   → runs the migration to create the actual SQL
tables
```

Our migration files are in `E-Grocery/Migrations/` and contain:

- `CreateTable` for Users, Products, CartItems, Orders
- `AddColumn` for fields like ProfilePicture and StockQty **Key point:** We write C# migration code, and EF Core generates the SQL to build our tables.

---

4. The Golden Rule — What Happens When You Click Submit

This is the exact order of events for **every form** in the app. Each step has a purpose:

```
User fills the form and clicks Submit
        ↓
[1] Client-Side Validation
    The browser checks required fields, email format, password length, etc.
    This happens instantly — no data sent to the server yet.
    Powered by jQuery Validation from _ValidationScriptsPartial.
        ↓
    IF valid → the POST request travels to the server
    IF invalid → errors show immediately, form does NOT submit
        ↓
[2] Model Binding
    ASP.NET receives the POST data and automatically creates a ViewModel object.
    It matches form input names to ViewModel properties by name.
    Example: input name="Email" → model.Email = "user@example.com"
        ↓
```

```
[3] Server-Side Validation
    ModelState.IsValid checks all [DataAnnotation] rules on the ViewModel.
    This is the SECURITY GUARD — it runs BEFORE any database code.
        ↓
    IF invalid → return View(model) with error messages.
             → The database is NEVER touched. Invalid data cannot reach SQL
Server.
    IF valid → continue to database code.
        ↓
[4] Database Operations
    The controller creates or updates database model objects.
    EF Core tracks these changes in memory.
        ↓
[5] SaveChanges()
    EF Core generates and executes the actual SQL INSERT / UPDATE / DELETE.
    This is the ONLY point where the database is modified.
        ↓
[6] Redirect or Return View
    Success → RedirectToAction("SomePage") — prevents duplicate submits on
refresh.
    Fail    → return View(model) with errors.
```

Key defense point: Validation is a **two-layer defense**. Client-side catches mistakes instantly for a better user experience. Server-side is the **real security** — even if someone disables JavaScript or sends fake POST data, the controller still validates before touching the database.

---

5. Account Registration Flow

GET — User Opens `/Account/Register`

1. Browser sends a GET request to the server.
2. Routing maps it to `AccountController.Register()` — the version with **no parameters**.
3. The controller runs `return View();` which tells ASP.NET to render `Views/Account/Register.cshtml`.
4. The view is rendered with an **empty** `RegisterViewModel`, so all fields start blank.

POST — User Clicks "Create Account"

What happens step by step:

The form has `method="post"` and `asp-action="Register"`, so the browser collects all input values and sends them as a POST request to `/Account/Register`.

ASP.NET sees the action expects `RegisterViewModel model`, so it automatically creates a new object and fills it from the form data. This is **Model Binding**.

Then `ModelState.IsValid` checks the validation rules:

- `[Required]` — did the user leave any field empty?
- `[EmailAddress]` — is the email in a valid format?

- `[StringLength]` — is the name 2–100 characters? Is the password at least 6?
- `[Compare("Password")]` — does ConfirmPassword match Password?

If validation FAILS: The controller hits `return View(model);` immediately. The same page reloads with the user's data still in the fields, and `<span asp-validation-for="...">` displays the error messages next to each invalid field. **The database is never touched.**

If validation PASSES: The controller creates a `User` object (the database model, NOT the ViewModel) and copies the validated data over:

```
var user = new User
{
    FullName = model.FullName,
    Email = model.Email,
    Password = model.Password
};
```

Notice: `ConfirmPassword` is NOT copied because it only exists in the ViewModel for form validation. It never gets saved to the database.

Then the database operations run:

```
_context.Users.Add(user);      // EF Core queues an INSERT
_context.SaveChanges();        // EF Core generates SQL and runs it
```

Finally: `return RedirectToAction("Login");` — the browser receives an HTTP redirect and navigates to the login page.

---

6. Login Flow

GET — User Opens `/Account/Login`

- Same pattern as Register GET: empty form rendered from `LoginViewModel`.

POST — User Clicks "Login"

Data flow:

1. Form values bind to `LoginViewModel` automatically.
2. `ModelState.IsValid` checks `[Required]` and `[EmailAddress]`.
3. If validation passes, the controller fetches ALL users from the database:

   ```
   var users = _context.Users.ToList();
   ```

   We use `ToList()` because our class notes teach us to retrieve all records first, then loop.

4. A `foreach` loop compares email AND password:

```
foreach (var u in users)
{
    if (u.Email == model.Email && u.Password == model.Password)
    {
        user = u;
        break;
    }
}
```

5. **If found:** Save email to session, redirect to store:

```
HttpContext.Session.SetString("UserEmail", user.Email);
return RedirectToAction("Index", "Store");
```

6. **If NOT found:** Add a custom error and return the view:

```
ModelState.AddModelError("", "Invalid email or password");
return View(model);
```

The empty string `""` means the error is not attached to a specific field — it appears at the top of the form via `asp-validation-summary="ModelOnly"`.

---

7. Profile Update Flow

GET — User Clicks "My Account"

1. The controller gets the user's email from session:

```
var email = HttpContext.Session.GetString("UserEmail");
```

2. If session is empty → redirect to Login (user is not logged in).
3. Gets all users with `_context.Users.ToList()`.
4. Loops with `foreach` to find the matching email.
5. Returns `Profile.cshtml` with the `User` object so the form is pre-filled.

POST — User Clicks "Save Changes"

1. The form must have `enctype="multipart/form-data"` because it includes a file upload.
2. Model binding fills `User model` (name) and `IFormFile profilePicture` (the uploaded file).
3. Session check again — if not logged in, redirect to Login.
4. Find the user in the database using `ToList()` + `foreach`.

5. Update the name: `user.FullName = model.FullName;`
6. **If a picture was uploaded:**

```
if (profilePicture != null && profilePicture.Length > 0)
{
    // Extract file extension
    var ext = "";
    var originalName = profilePicture.FileName;
    if (originalName.Contains("."))
    {
        ext = originalName.Substring(originalName.LastIndexOf("."));
    }

    // Create unique filename using DateTime.Now.Ticks
    var fileName = DateTime.Now.Ticks.ToString() + ext;
    var filePath = Directory.GetCurrentDirectory() +
"\\wwwroot\\images\\profiles\\" + fileName;

    // Save file to disk using FileStream inside a using block
    using (var stream = new FileStream(filePath, FileMode.Create))
    {
        profilePicture.CopyTo(stream);
    }

    // Store the relative path in the database
    user.ProfilePicture = "/images/profiles/" + fileName;
}
```

7. `_context.SaveChanges();` writes the updated name and picture path to the database.
8. `return RedirectToAction("Profile");` reloads the page so the user sees their updated info.

Key point: The actual image file is stored on the **server disk** (`wwwroot/images/profiles/`). Only the **file path** is stored in the database. The view displays it with `<img src="@Model.ProfilePicture" />`.

---

8. Store / Browse Products Flow

GET — `/` or `/Store/Index`

1. `StoreController.Index()` fetches all products:

```
var products = _context.Products.ToList();
```

2. Returns them to `Views/Store/Index.cshtml`:

```
return View(products);
```

3. The view loops through the list:

```
@foreach (var product in Model)
{
    // Display image, name, description, price
}
```

4. Price is formatted as Philippine Peso: `₱@product.Price.ToString("N2")`
5. Each product card opens a modal where the user picks a quantity.

---

9. Search Products Flow

GET — `/Store/Search?searchTerm=apple`

1. The search form in the header uses `method="get"` and sends the term in the URL.
2. `StoreController.Search(string searchTerm)` receives it automatically via **model binding from the query string**.
3. Gets all products: `_context.Products.ToList()`
4. Checks if `searchTerm` is null or empty:
    - If yes → return ALL products (same as browsing)
    - If no → loop through every product and check:

```
if (p.Name != null && p.Name.ToLower().Contains(term.ToLower()))
```

5. Matching products are added to a new `List<Product>` called `filtered`.
6. Returns the **same** `Index.cshtml` view with the filtered list:

```
return View("Index", filtered);
```

Why not `Where()`? Our class notes teach `ToList()` + `foreach` for filtering. We follow what was taught.

---

10. Add to Cart Flow

POST — User Clicks "Add to Cart"

1. The modal form POSTs `productId` and `quantity` to `StoreController.AddToCart()`.
2. Model binding automatically fills the action parameters from the form.
3. Find the product by ID:

```
var products = _context.Products.ToList();
Product product = null;
foreach (var p in products)
```

```
{
    if (p.Id == productId) { product = p; break; }
}
```

4. Check if this product is already in the cart:

```
var cartItems = _context.CartItems.ToList();
CartItem existing = null;
foreach (var c in cartItems)
{
    if (c.ProductId == productId) { existing = c; break; }
}
```

5. **If already in cart:** Just increase the quantity:

```
existing.Quantity += quantity;
```

6. **If NOT in cart:** Create a new `CartItem` with the product's details copied in:

```
_context.CartItems.Add(new CartItem
{
    ProductId = product.Id,
    ProductName = product.Name,
    Price = product.Price,
    ImageUrl = product.ImageUrl,
    Quantity = quantity
});
```

7. `_context.SaveChanges();` writes the change to the database.
8. `return RedirectToAction("Cart");` takes the user to the cart page.

Important: Cart data lives in the **database** (`CartItems` table), not in the browser. This means the cart survives page refreshes and is shared across devices if the user is logged in.

---

11. View Cart Flow

GET — `/Store/Cart`

1. `StoreController.Cart()` gets all cart items:

```
var cartItems = _context.CartItems.ToList();
```

2. Calculates the grand total by looping:

```
decimal total = 0;
foreach (var item in cartItems)
{
    total += item.Price * item.Quantity;
}
ViewData["Total"] = total;
```

3. Returns the items to `Cart.cshtml`:

```
return View(cartItems);
```

4. The view displays each item with its subtotal (`Price * Quantity`) and shows the grand total from `ViewData["Total"]`.

---

12. Place Order Flow

POST — User Clicks "Place Order"

1. Get the user's email from session. If not logged in, use `"Guest"`:

```
var userEmail = HttpContext.Session.GetString("UserEmail");
if (userEmail == null || userEmail == "")
{
    userEmail = "Guest";
}
```

2. Get all cart items:

```
var cartItems = _context.CartItems.ToList();
```

3. For EACH item in the cart, create an `Order` record:

```
foreach (var item in cartItems)
{
    var order = new Order
    {
        UserEmail = userEmail,
        ProductName = item.ProductName,
        Price = item.Price,
        Quantity = item.Quantity,
        Total = item.Price * item.Quantity,
        OrderDate = DateTime.Now
    };
```

```
    _context.Orders.Add(order);
}
```

Each cart item becomes one row in the `Orders` table.

4. Clear the cart by removing all `CartItem` records:

```
foreach (var item in cartItems)
{
    _context.CartItems.Remove(item);
}
```

5. **One** `SaveChanges()` saves everything at once:

```
_context.SaveChanges();
```

EF Core generates SQL INSERTs for all the new orders AND SQL DELETEs for all cart items in a single transaction.

6. `return View("OrderComplete");` shows the "Order Complete!" success page.

---

13. Order History Flow

GET — `/Store/OrderHistory`

1. Get email from session (fallback to `"Guest"` if not logged in).
2. Get all orders: `_context.Orders.ToList()`
3. Filter by email using `foreach`:

```
var userOrders = new List<Order>();
foreach (var o in allOrders)
{
    if (o.UserEmail == userEmail)
    {
        userOrders.Add(o);
    }
}
```

4. Return to `OrderHistory.cshtml` which displays product name, date, quantity, price, and total for each order.

---

14. Admin Product Management Flow

Create — GET `/Admin/Create`

This page shows a form at the top AND a list of existing products below it.

```
ViewData["Products"] = _context.Products.ToList();      // list for display
ViewData["CartCount"] = _context.CartItems.ToList().Count;
ViewData["HideNav"] = true;                              // hides search bar on
this page
var model = new ProductCreateViewModel();               // empty form
return View(model);
```

Create — POST (Admin clicks "Save Product")

1. `ModelState.IsValid` checks `[Required]`, `[StringLength]`, `[Range]` on the `ProductCreateViewModel`.
2. **If an image file was uploaded:**

```
if (model.ProductImage != null && model.ProductImage.Length > 0)
{
    var ext = "";
    var originalName = model.ProductImage.FileName;
    if (originalName.Contains("."))
    {
        ext = originalName.Substring(originalName.LastIndexOf("."));
    }
    var fileName = DateTime.Now.Ticks.ToString() + ext;
    var filePath = Directory.GetCurrentDirectory() +
"\\wwwroot\\images\\products\\" + fileName;

    using (var stream = new FileStream(filePath, FileMode.Create))
    {
        model.ProductImage.CopyTo(stream);
    }

    imagePath = "/images/products/" + fileName;
}
```

3. **Check if Create or Update:**
   - `model.Id == 0` → this is a **NEW** product:

```
var product = new Product { ... };
_context.Products.Add(product);
```

   - `model.Id > 0` → this is an **UPDATE**:

```
// Find existing product by Id
// Update each field
```

```
// SaveChanges()
```

4. `_context.SaveChanges();` commits the change.
5. `return RedirectToAction("Create");` reloads the page so the new product appears in the list.

Edit

- Admin clicks **Edit** on a product → goes to `/Admin/Edit/5`
- `Edit(int id)` finds the product, copies data into `ProductCreateViewModel`, and returns `View("Create", model)`
- This reuses the **same** Create view, but the form is pre-filled with existing data
- Admin changes data and submits → `model.Id > 0` triggers the UPDATE branch

Delete

- Admin clicks **Delete** → confirmation modal appears
- Confirm → POST to `AdminController.Delete(int id)`
- Find product with `ToList()` + `foreach`
- `_context.Products.Remove(product)` → `SaveChanges()` → redirect

---

15. Validation: Two Layers Explained

Layer 1 — Client-Side (Browser)

- Our views include `@section Scripts { <partial name="_ValidationScriptsPartial" /> }`
- This loads jQuery Validation into the browser
- When the user clicks Submit, the browser checks all rules **first**
- If invalid → instant red error messages, **no network request sent**
- **Purpose:** Better user experience (fast feedback)

Layer 2 — Server-Side (Controller)

- `ModelState.IsValid` runs in the controller action
- It checks the `[DataAnnotation]` attributes on the ViewModel
- If invalid → `return View(model)` with errors, **database untouched**
- **Purpose:** Security — even if someone bypasses the browser, the server still protects the database

Defense Q: "What if the user disables JavaScript?" **A:** "Client-side validation won't run, but server-side validation still catches every error. The database is protected because `ModelState.IsValid` is a guard that runs before any database code."

---

16. Routing

Configured in `Program.cs`:

```
app.MapControllerRoute(
    name: "Default",
```

```
    pattern: "{controller}/{action}/{id?}",
    defaults: new { controller = "Store", action = "Index" });
```

URL	Goes To
`/`	`StoreController.Index()`
`/Account/Login`	`AccountController.Login()`
`/Account/Register`	`AccountController.Register()`
`/Store/Cart`	`StoreController.Cart()`
`/Store/OrderHistory`	`StoreController.OrderHistory()`
`/Admin/Create`	`AdminController.Create()`
`/Admin/Edit/5`	`AdminController.Edit(id: 5)`

The `id?` means the `id` parameter is **optional** — you can visit `/Admin/Create` without an ID.

---

17. Common Defense Questions & Answers

Q: "What is MVC?" A: Model (data), View (UI), Controller (requests). The Controller receives input, uses the Model to get/save data, then picks a View to display results.

Q: "What is Entity Framework Core?" A: An ORM (Object-Relational Mapper). We work with C# objects, and EF Core translates our `_context.Add()`, `_context.SaveChanges()` into actual SQL INSERT, UPDATE, DELETE commands.

Q: "What does `DbContext` do?" A: It's the bridge between our C# code and the SQL Server database. It tracks our objects and knows what SQL to generate when we call `SaveChanges()`.

Q: "What is `ModelState.IsValid`?" A: It checks if all validation attributes on the ViewModel passed. If false, the form redisplays with error messages and the database code is **skipped entirely**.

Q: "What triggers first — validation or database?" A: **Validation first.** For every form, `ModelState.IsValid` runs before any database code. If validation fails, the controller returns the view immediately and the database is never touched. Invalid data cannot reach SQL Server.

Q: "What is Model Binding?" A: It's how ASP.NET automatically converts form values into C# objects. When the browser POSTs `Email=john@example.com`, ASP.NET creates a `LoginViewModel` and sets `model.Email = "john@example.com"` for us.

Q: "Why `ToList()` + `foreach` instead of `Find()` or `Where()`?" A: Our class notes teach us to get all records with `ToList()` and loop with `foreach`. We follow exactly what was taught.

Q: "How does the cart work?" A: Cart items are stored in the `CartItems` database table. When you add a product, it creates a `CartItem` record. When you place an order, all `CartItem` records are deleted and `Order` records are created.

Q: "What is the difference between `User` and `RegisterViewModel`?" A: `User` is the database entity — it has only fields that belong in the database. `RegisterViewModel` is only for the form — it has extra fields like `ConfirmPassword` and validation rules that don't exist in the database table.

Q: "How does file upload work?" A: The form uses `enctype="multipart/form-data"`. The controller accepts `IFormFile`. We extract the file extension, generate a unique filename using `DateTime.Now.Ticks`, save the file with `FileStream` inside a `using` block, and store the relative path in the database.

Q: "What is the purpose of `RedirectToAction` after saving?" A: It sends an HTTP 302 redirect to the browser. This prevents the "duplicate form submission" problem — if the user refreshes the page after saving, they just reload the target page instead of resubmitting the form.

Q: "What is a ViewModel?" A: A class made specifically for a form or view. It has validation rules and extra fields that the database model doesn't need. Example: `RegisterViewModel` has `ConfirmPassword`, but the `User` table does not.

Q: "What is `[ValidateAntiForgeryToken]`?" A: It protects against Cross-Site Request Forgery (CSRF) attacks. It ensures the POST request came from our own form, not from a fake website. Every POST action has this attribute.

Q: "What is `IFormFile`?" A: It's the ASP.NET class that represents an uploaded file. Our `Profile` action and `ProductCreateViewModel` both use it to receive images from the browser.

Q: "Why does the form need `enctype="multipart/form-data"`?" A: Normal forms only send text. To send a file (like a profile picture), the form must use `multipart/form-data` so the browser can attach the binary file data.

Q: "What are Data Annotations?" A: Attributes like `[Required]`, `[EmailAddress]`, `[StringLength]`, and `[Compare]` that we place on ViewModel properties. They define validation rules that are checked by both client-side and server-side validation.

Q: "What is a `DbSet`?" A: It represents a database table inside our `DbContext`. Example: `public DbSet<User> Users { get; set; }` means EF Core will create and manage a `Users` table in SQL Server.

Q: "What is a Migration?" A: A C# file that describes how to create or modify database tables. We run `Add-Migration` to create it and `Update-Database` to apply it. It lets us build the database schema using C# code instead of writing SQL by hand.

Q: "What is the `using` block in file upload?" A: `using` automatically closes and cleans up the `FileStream` after saving the file. It's good practice because it frees memory and prevents the file from staying locked.

Q: "What is Dependency Injection?" A: It's how ASP.NET automatically gives us the `AppDbContext`. We declare `private readonly AppDbContext _context;` in the constructor, and the framework creates and passes it for us. We never manually create the database connection.

Q: "What is Razor?" A: The syntax in `.cshtml` files that mixes HTML with C# code. It uses `@` to switch from HTML to C#: `@Model.Email`, `@foreach`, `@if`, etc.

Q: "What does `asp-for` do?" A: It's a Tag Helper that connects a form input to a ViewModel property. Example: `<input asp-for="Email">` generates the correct `name`, `id`, and validation attributes automatically.

Q: "What does `asp-action` do?" A: It's a Tag Helper that sets the form's action URL. Example: `<form asp-action="Register">` tells the form to POST to `/Account/Register`.

Q: "What does `asp-validation-for` do?" A: It displays the error message for a specific field. Example: `<span asp-validation-for="Email">` shows "Email is required" if the user leaves it blank.

Q: "What is `ViewData`?" A: A dictionary that passes extra data from the controller to the view that is not part of the main model. Example: `ViewData["Total"]` passes the cart total to the view.

Q: "What is `_Layout.cshtml`?" A: The master page template. Every view uses it automatically. It contains the `<head>`, navigation header, footer, and shared scripts. The actual page content is inserted where `@RenderBody()` is placed.

Q: "What is the difference between GET and POST?" A: GET is for retrieving data — it shows a page or list (safe, no changes). POST is for submitting data — it creates, updates, or deletes records. Our forms use POST because they change the database.

Q: "What is `return View(model)`?" A: It tells ASP.NET to render the matching `.cshtml` file and pass the model object to it. The view then uses `@Model` to display the data.

Q: "What is `_ValidationScriptsPartial`?" A: A partial view that loads jQuery Validation scripts. Views include it with `@section Scripts { <partial name="_ValidationScriptsPartial" /> }` to enable client-side validation.

Q: "What is `DateTime.Now.Ticks`?" A: It returns a unique long number based on the current time. We use it to create unique filenames for uploaded images so files never overwrite each other.

Q: "What is `FileStream`?" A: A class that opens a file on the server disk for reading or writing. We use it with `FileMode.Create` to save uploaded images to `wwwroot/images/`.

Q: "What are `[HttpGet]` and `[HttpPost]`?" A: They tell ASP.NET which HTTP method an action responds to. `[HttpGet]` shows the empty form. `[HttpPost]` handles the form submission. Without them, the action responds to any HTTP method.

Q: "What is conventional routing?" A: The pattern `{controller}/{action}/{id?}` in `Program.cs`. The URL `/Store/Cart` maps to `StoreController.Cart()`. The `id?` part is optional.

Q: "What is `wwwroot`?" A: The public folder in ASP.NET. Files inside it (images, CSS, JS) are directly accessible by the browser via URL. Uploaded images go to `wwwroot/images/`.

---

18. Class Notes Compliant Syntax

We Use (Notes-Compliant)	We Avoid (Advanced)	Reason
`ToList()` + `foreach` loops	LINQ `Where()`, `FirstOrDefault()`, `Find()`	Notes teach `ToList()` + `foreach`
`if (x == null || x == "")`	`string.IsNullOrEmpty(x)`	Notes don't cover `IsNullOrEmpty`
`if (x == null)` manual checks	`x ?? y` (null-coalescing operator)	Notes don't cover `??`

We Use (Notes-Compliant)	We Avoid (Advanced)	Reason
`IActionResult` (synchronous)	`async Task<IActionResult>`	Notes don't cover `async/await`
`DateTime.Now.Ticks.ToString()`	`Guid.NewGuid()`	Notes don't cover `Guid`
`string` initialized to `string.Empty`	`string?` (nullable reference types)	Notes don't cover nullable reference types
`ViewData["Key"]`	`ViewContext.RouteData.Values`	Notes don't cover `ViewContext`
Manual null checks in views	`@Html.Raw()` or complex helpers	Keeping it simple per notes

Key defense point: Every pattern we use comes directly from our class notes. We do not use advanced C# features that were not taught.

---

19. File Map

File	Purpose
`Program.cs`	App startup, routing, DB connection, session config
`appsettings.json`	Connection string
`Data/AppDbContext.cs`	DbContext with `DbSet<Users>`, `DbSet<Products>`, `DbSet<CartItems>`, `DbSet<Orders>`
`Models/User.cs`	Database model: `Id`, `FullName`, `Email`, `Password`, `ProfilePicture`
`Models/Product.cs`	Database model: `Id`, `Name`, `Description`, `Price`, `ImageUrl`, `StockQty`
`Models/CartItem.cs`	Database model: `Id`, `ProductId`, `ProductName`, `Price`, `Quantity`, `ImageUrl`
`Models/Order.cs`	Database model: `Id`, `UserEmail`, `ProductName`, `Price`, `Quantity`, `Total`, `OrderDate`
`Models/RegisterViewModel.cs`	Form validation: FullName, Email, Password, ConfirmPassword
`Models/LoginViewModel.cs`	Form validation: Email, Password
`Models/ProductCreateViewModel.cs`	Form validation: Name, Description, Price, StockQty, Image, ProductImage
`Controllers/AccountController.cs`	Login, Register, Logout, Profile GET/POST

File	Purpose
Controllers/StoreController.cs	Index, Search, Cart, AddToCart, PlaceOrder, OrderHistory
Controllers/AdminController.cs	Create (GET/POST), Edit, Delete
Views/Account/Register.cshtml	Registration page with validation
Views/Account/Login.cshtml	Login page with validation
Views/Account/Profile.cshtml	Profile edit + picture upload
Views/Store/Index.cshtml	Product listing / search results
Views/Store/Cart.cshtml	Shopping cart with totals
Views/Store/OrderComplete.cshtml	Order success message
Views/Store/OrderHistory.cshtml	Past orders list
Views/Admin/Create.cshtml	Admin product form + product list
Views/Shared/_Layout.cshtml	Master page: header, nav, search bar, footer
Views/Shared/_ValidationScriptsPartial.cshtml	Loads jQuery Validation scripts for client-side validation
Views/Shared/Error.cshtml	Error page (scaffolded by ASP.NET)
Controllers/HomeController.cs	Scaffolded home page (not used in our app flow)
Migrations/	EF Core migration files that built our database tables